N. Sanjay

192524148

ITA0612

Machine Learning

CO2 - AT3 Debugging and

optimization Task.

CO2 - AT3

Q1 A perception model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence.

Problem :- A perception trained on a linearly separable dataset fails to converge even after many iterations.

possible causes :-

1) Improper learning rate : A very high learning rate may cause unstable weight updates, while a very low rate may slow learning.

2) Incorrect weight initialization : Improper initial weights may affect the initial learning process.

3) Bias handling errors :- If the bias term is missing or incorrectly updated, the decision boundary may be wrong.

4) Incorrect update rule :- Errors in calculating prediction errors or updating weights can prevent convergence.

Debugging & optimizations:

- check the dataset and confirm that it is linearly separable.
- Verify the weight and bias update equations
- Test the algorithm using a small dataset with known outputs.
- Adjust the learning state and monitor errors after each iteration.

Q2. While training a multilayer perception using the backpropagation algorithm, the network exhibits slow convergence and gets struck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adapting learning rates, normalization, or advanced optimizers to improve performance.

problem :- A multilayer perceptron (MLP) trained using backpropagation learns slowly and gets stuck in poor solutions

possible causes:

1) Vanishing gradients : Gradients become very small in early layers, preventing effective learning.
2) Inappropriate activation functions : Sigmoid or tanh may saturate and slow learning.
3) Poor weight Initialization : Improper weights can make training slow
4) Suboptimal learning rate : A very high or low learning rate affects convergence.

Debugging & optimization:

- Monitor training and validation loss after every epoch.
- check gradients to identify vanishing or exploding gradients.
- Use ReLU or other suitable activation functions
- Initialize weights using Xavier.
- Normalize input features and use optimizers such as Adam or RMSprop.

A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed.

problem:- A genetic algorithm used for hypothesis space search produces inconsistent and suboptimal solutions across different runs.

~~problem~~

possible causes:

1) Improper chromosome encoding: The representation may not correctly represent possible solutions.

2) Incorrect fitness function: The fitness function may not evaluate solution quality properly.

3) Premature convergence: The population may become similar too early.

4) Lack of diversity: Insufficient mutation may Prevent exploration of new solutions.

Debugging & optimization:

- Verify chromosomes encoding and decoding.
- Test the fitness function using known solutions
- Monitor population diversity and best fitness across generations.
- Adjust the mutation rate to mantain diversity.
- Balance selection pressure and use elitism to preserve the best solutions

Q+1 A deep neural network combined with genetic programming is used for solving a complex prediction problem, but it suffers from overfitting and high computational cost identify debugging steps to analyze issues related to model completely, over-parameterization, and insufficient training data. propose optimization techniques such as regularization, dropout, pruning, and efficient evaluation models to balance accuracy and computational efficiency while improving generalization capability.

Problem : A deep neural network combined with genetic programming has high computational cost and performs poorly on unseen data.

Possible causes:

1) Model complexity : An overly complex model may learn noise instead of general patterns.
2) Over-Parameterization : Too many parameters increase computation and risk of overfitting.
3) Insufficient training data : Limited data prevents the model from learning general patterns.
4) Expensive evaluation : Evaluating many candidate programs increases computational cost.

Debugging & optimization:

- Compare training accuracy with validation accuracy to identify overfitting.
- Apply L_1 or L_2 regularization to reduce excessive model complexity.
- Use dropout to prevent excessive dependence on individual neurons.
- prune unnecessary network connections and genetic program branches.
- Use efficient fitness evaluation, early stopping, and cross-validation.

decision tree model trained on a large dataset shows high accuracy on traning data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. propose debugging steps and optimization methods like Pruning, cross-validation, and feature selection to improve generalization.

Problem :- A decision tree performs well on training data but poorly on unseen data.

Possible causes :-

1) Overfitting : The tree memorizes training examples instead of learning general patterns.
2) Deep tree structure : Too many branches and leaves make the model complex.
3) Noisy data : Incorrect labels or irrelevant features affect the tree.
4) poor feature selection : Unnecessary features may lead to unreliable splits.

Debugging & optimization :

- Compare training and test accuracy to identify overfitting.
- Use cross-validation to evaluate performance on different data subsets.
- Apply Pre-pruning by limiting tree depth and minimum samples per leaf.
- Apply post-pruning to remove unnecessary branches.
- perform feature selection and clean noisy data